

Beginner Overview — webRagSys_Django 初心者向け解説

0. この文書の読み方

この文書は、Pythonの基本文法（変数、関数、クラス、辞書、リスト）を一通り書いたことがある人が、初めてこのシステムの中身を読むための案内である。RAG（あとで説明する検索と回答の仕組み）の知識は前提としない。専門用語は出てきた順に、その場で短く説明する。

たとえるなら、文書管理は書庫係、RAGは索引係、LLMは回答係である。書庫係が紙をしまい、索引係が索引カードを作り、回答係が索引を見て答える。技術的に正確に言い直すと、文書の保存はDjango ORM、分割と検索の手順はLangChain由来の自前層、文章生成はLLMプロバイダが担う。

読み方の推奨は、1章から順に読むことである。4章の具体例が中心であり、3章までの用語は4章で実物と結び付く。

1. このシステムは何をするのか

1.1 目的

社内の文書（例：休暇規定）を登録し、あとから「休暇は何日ですか」と質問すると、登録した文書を根拠にAIが答えるWeb APIシステムである。

できることは6件である。

操作	経路	意味
生存確認	GET /health	応答の有無
一覧	GET /documents	登録済みの一覧
詳細	GET /documents/数字	1件の取出し
登録	POST /documents	新規の保存と索引作り
更新	PUT /documents/数字	上書きと索引作り直し
削除	DELETE /documents/数字	本体と索引の削除
質問	POST /chat	索引検索とAI回答

1.2 なぜDjangoか

処理の本質は「受付→保存→検索→生成」の順である。Djangoが担うのは受付と保存だけである。検索と生成にDjangoの相当機能はなく、そこはLangChain由来の層で担う。

置き換えの要点は次のとおりである。詳細はREADMEの置き換え表を参照する。

参考の部品	本件の部品	役割の変化
FastAPI	Django + DRF	受付と振分の担当交代
Pydantic	Serializer	入出力検品の担当交代
SQLAlchemy	Django ORM	保存読出の担当交代
uvicorn	runserver	起動手段の交代
/docs	/api/docs/	操作画面の場所の交代

変えないものは、APIの経路、JSONの項目名、偽実装既定、SQLite正規、索引の作り直し規則である。

2. 事前に知る用語

ここでは後の章で使う言葉だけを、やさしい順に説明する。丸暗記は不要であり、「そんなものがある」と分かればよい。

2.1 APIとJSON

API（エーピーアイ、Application Programming Interface）は、プログラム同士の受付窓口である。ブラウザの画面ではなく、JSON（ジェイソン、JavaScript Object Notation）という文字形式で注文と受取りを行う。注文の例は `{"title": "休暇規定", "content": "年次休暇は10日付与する。"}` であり、波括弧で囲んだ名前と値の組である。

2.2 CRUD

CRUD（クラッド）はCreate（登録）、Read（読出）、Update（更新）、Delete（削除）の頭文字である。本システムの文書操作そのものである。

2.3 RAG

RAG（ラグ Retrieval-Augmented Generation、検索拡張生成）は、先に文書を探してから答える方式である。AIが何も見ずに答えるのではなく、登録文書の該当部分を机に広げてから答える。手順は3段階である。

- 1. RAG化：文書を小さく切り、探せる形で保存する
- 2. 検索：質問に似た切り身を探す
- 3. 生成：見つかった切り身を添えてAIに文章を作らせる

2.4 チャンクとEmbeddingとベクトル

チャンクは、長い文書を切った1枚1枚である。本件では500文字ごとに切る（`chunk_size=500`）。切り口は50文字重なる（`chunk_overlap=50`）。重なる理由は、切れ目で文が分断されても前後が残るためである。

Embedding（エンベディング）は、文を数の列に変換することである。ベクトルは、その数の列である。例として「年次休暇は10日付与する。」の偽ベクトルは次の8個の数である。

```
[0.34, 0.47, 0.60, 0.73, 0.86, 0.99, 0.12, 0.25]
```

本物は外部APIが作る意味のこもった数列であるが、学習時は課金を避けるため、文字から決定的に作る偽物で代用する（`rag/embeddings.py:21-36`）。決定的とは、同じ文からは必ず同じ数列が出る性質である。

2.5 LLM

LLM（エルエルエム、Large Language Model、大規模言語モデル）は、文章を作るAIである。本件では既定で偽物を使い、「[FAKE回答] 質問「...」に対する関連情報：「...」」という定型文を返す（`llm/fake_provider.py:17-28`）。本番だけ外部APIに切り替わる（`llm/provider.py:19-44`）。

2.6 Djangoの言葉

DjangoはWebアプリ作りの道具一式である。関連語は次のとおりである。

言葉	意味
アプリ	機能の束。文書束と質問束の2束構成
モデル	テーブルの設計図。属性を書くと表になる
ORM	表操作の翻訳係。SQL直書きの代替手段
マイグレーション	設計図の変更履歴。適用で表ができる
Serializer	入出力の検品係。検証と整形の担当
view	受付の裏方。注文を受けて応える関数
urls	振分表。経路と裏方の対応表

3. フォルダ構成と主要ファイル

webRagSys_Django/

- ├── manage.py
- ├── config.py
- └── ragproject/

- ← 起動と管理の命令口
- ← 設定集約（.env読込）
- ← 本体（settings、urls、例外変換）

└── documents/	← 文書CRUD (models、serializers、views、urls、admin)
└── chat/	← 質問API (serializers、views、urls)
└── rag/	← chunker、embeddings、vectorstore、chain
└── llm/	← provider.py (切替)、fake_provider.py (偽回答)
└── tests/	← test_documents.py、test_chat.py
└── docs/	← 本書、handover.md
└── diagrams/	← 図6種

ファイル	担当	要点
ragproject/settings.py	設定	SQLite指定 (71-72行目)、末尾スラッシュ無効 (88行目)、例外変換指定 (93行目)
ragproject/urls.py	振分	health、documents、chat、api/docs (28-32行目)
ragproject/exceptions.py	変換	検証失敗400を422に直す
documents/models.py	表定義	Document (16-36行目)、DocumentChunk (39-59行目)
documents/serializers.py	検品整形	登録更新用と応答用の2種
documents/views.py	文書裏方	検証・本体・整形の3役分担
chat/views.py	質問裏方	chainへの委譲と500処理 (82-92行目)
rag/chunker.py	分割	LangChain優先、なければ単純分割
rag/embeddings.py	数列化	Fake既定、本番のみ外部通信
rag/vectorstore.py	保存検索	置換保存、コサイン検索
rag/chain.py	手順決定	検索→整形→生成の順序だけを定める
llm/provider.py	切替	Fakeか本番かを1か所で決める

4. 具体例：1件の文書が答えになるまで

題名「休暇規定」、本文「年次休暇は10日付与する。」の13文字で追う。短いため分割後は1枚である。

4.1 登録（外部 → DB）

POST /documents にJSONを送ると、documents/views.py:93-113 の document_list_create が動く。流れは次のとおりである。

1. DocumentWriteSerializer が検証する (107-109行目)。題名200文字以内、本文1文字以上、空白のみ不可である
2. validate_input が空白を再確認する (111行目)。二重確認の理由は、Serializerの規則と業務規則を分けるためである
3. execute_logic_register が保存する (112行目、40-56行目)。Document.objects.create が1行作り、直後に索引作りが走る

応答は201であり、本体は {"id": 1, "title": "休暇規定", ...} である。201の意味は「作った」である。

4.2 RAG化（文書 → ベクトル）

登録直後の vectorstore.save_chunks(doc.id, chunker.split_text(doc.content)) (55行目) が3段階で動く。

1. 分割：split_text が本文を切る。13文字は500文字未満のため1枚のままである
2. 数列化：embed_texts が偽ベクトルを作る。結果は [0.34, 0.47, 0.60, 0.73, 0.86, 0.99, 0.12, 0.25] である
3. 保存：save_chunks (rag/vectorstore.py:41-58) が古い索引を消してから新しい行を入れる。置換の理由は、更新時に古い意味が残らないためである

DocumentChunk の1行は、文書番号1、枝番0、本文、数列の4点組である。

4.3 質問応答（質問 → 回答）

POST /chat に {"question": "休暇は何日ですか。"} を送ると、chat/views.py:66-93 の chat が動く。

1. ChatRequestSerializer が検証する (76-78行目)。空質問は422である

2. `execute_logic` が `answer_question` に委ねる（83行目、40-49行目）
3. `vectorstore.search`（73-90行目）が質問を数列表化し、全索引との似具合を計算する。質問の偽ベクトルは `[0.02, 0.15, 0.28, 0.41, 0.54, 0.67, 0.80, 0.93]` であり、本文との似具合は0.7253である。上位3件（`top_k=3`）を採る
4. `generate_answer` が偽回答を作る。本文先頭200文字を埋めた定型文である

応答は次のとおりである。

```
{"answer": "[FAKE回答] 質問「休暇は何日ですか。」に対する関連情報：「年次休暇は10日付与する。」", "source_ids": [1]}
```

`source_ids` の意味は根拠文書の番号一覧である。未登録で質問すると空になる。

4.4 コサイン類似度の直感

コサイン類似度は、2つの矢印の向きの近さである（`rag/vectorstore.py:25-38`）。1に近いほど同じ向き、すなわち似た文である。長さではなく向きで比べるため、文の長短に引っ張られない。たとえるなら、方角の近さ比べである。

5. 更新と削除で起きること

更新（`PUT /documents/1`）は `execute_logic_update`（59-78行目）が担う。本体を書き換えたあと、索引を全置換する（77行目）。差分更新ではない理由は、古い切り身と新しい切り身の対応付けが複雑なためである。丸ごと作り直す方が誤りが少ない。

削除（`DELETE /documents/1`）は索引削除が先である（139-140行目）。本体先行では、索引だけが残る事故が起きる。応答は204であり、本体なしの成功の意味である。

存在しない番号（例：99999）は404である（128行目の `get_object_or_404`）。`get_object_or_404` の意味は「なければ404を返す取出し」である。

6. Djangoでつまづきやすい点

6.1 アプリとは何か

Djangoのアプリは機能の束である。本件は `documents`（文書束）と `chat`（質問束）の2束である。`INSTALLED_APPS`（`ragproject/settings.py`）への登録が参加手続きである。登録漏れでは表も経路も動かない。

6.2 マイグレーションとは何か

モデル（設計図）を書いただけでは表はできない。`python manage.py migrate` の適用で表ができる。設計図と実物の差を埋める手続きである。`documents/migrations/0001_initial.py` が初回の履歴である。

6.3 Serializerとは何か

Serializerは検品係である。`DocumentWriteSerializer` が注文書の形式を確かめ、`DocumentOutSerializer` が渡す品を整える。Pydanticとの違いは、Django流の作法であることだけであり、役割は同じである。

6.4 なぜ末尾スラッシュなしなのか

Djangoは既定でURL末尾に斜線を足す癖がある。本件は `APPEND_SLASH=False`（88行目）で無効化し、参考のcurl互換を保つ。`/documents` と `/documents/` は別物として扱う。

6.5 なぜ422なのか

DRFの検証失敗は既定で400である。本件は `compat_exception_handler`（`ragproject/exceptions.py`）で422に直す。参考との学習内容を合わせるためである。400と422の違いは、前者が汎用の注文誤り、後者が内容の誤りである。

7. LangChainと偽実装でつまづきやすい点

7.1 LangChainはどこにいるのか

本件のLangChain利用は `langchain_text_splitters` だけである。`chunker.py` が `RecursiveCharacterTextSplitter` を使い、なければ単純分割に退避する。鎖状の高度機能（LCELなど）は使わない。将来作業のままである。

7.2 偽実装と本番の切替え

切替えは config.py の USE_FAKE と .env の OPENAI_API_KEY の2点で決まる。偽になる条件は「 USE_FAKE=true またはキー空」である（ rag/embeddings.py 、 llm/provider.py ） 。テストは全件偽で走り、外部通信しない。

7.3 誰が誰を使うのか

中心は rag/chain.py:answer_question である。検索（ vectorstore.search ） と生成（ llm/provider.py:generate_answer ） の順序だけを決める。LLMがRAGを使うのではなく、RAG手順がLLMを使う。下請け関係である。

主体	役割	実コード
chain.py	検索と生成の順序決定	answer_question
vectorstore.py	類似検索の実行	search
llm/provider.py	回答文生成の下請け	generate_answer

8. 操作と内部の対応（/api/docs/利用者向け）

8.1 画面の読み方

表示	内容
health	生存確認。最初に実行
documents	登録・一覧・詳細・更新・削除
chat	質問応答
schema	入出力雛形。参照用

基本操作は「展開 → Try it out → 入力 → Execute → Responses確認」である。

8.2 群別対応表

操作	画面の群	裏方の関数
生存確認	health	ragproject/urls.pyのhealth
一覧	documents	documents/views.py:document_list_create
詳細	documents	documents/views.py:document_detail
登録	documents	同上 + execute_logic_register
更新	documents	同上 + execute_logic_update
削除	documents	同上 + delete_chunks
質問	chat	chat/views.py:chat + answer_question

8.3 回答と異常の見分け方

現象	意味	対処
answerがFAKE定型文	偽実装動作中。正常	学習継続
source_idsが空	未登録で質問。正常	先に文書登録
404	番号の指し間違い	idの控えを確認
422	空入力等の内容誤り	入力の見直し

500	内部障害	ログ確認と再試行
-----	------	----------

付録：学習の順番

1. GET /health で生存を確認する
2. POST /documents で1件登録し、id を控える
3. GET /documents と GET /documents/1 で読出す
4. POST /chat で質問し、source_ids が1であることを確かめる
5. documents/models.py で表の形を読む
6. documents/views.py で検証・本体・整形の分担を読む
7. rag/chunker.py → embeddings.py → vectorstore.py の順に索引作りを読む
8. rag/chain.py と chat/views.py で質問の流れを読む
9. 図6種を眺め、手順全体を俯瞰する
10. PUT と DELETE で作り直しと削除を試す